



RESEARCH PROPOSAL

Evaluating Recommender Systems under Sparse User Feedback

Kamal Bhupendra Laloo
2652159

School of Computer Science and Applied Mathematics
Faculty of Science
University of the Witwatersrand

Supervisor: Hairong Bau

May 17, 2026

Declaration of Authorship

I, Kamal Bhupendra Lalloo, hereby declare that this research proposal is my own work. This proposal is submitted for the degree of Bachelor of Science with Honours in Computer Science at the University of the Witwatersrand. It has not been submitted to any other university or for any other degree.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Hairong Bau, for her guidance, advice, and continuous support throughout this project. Her feedback and insight played an important role in shaping the direction and development of this proposal.

Abstract

Evaluating Recommender Systems under Sparse User Feedback

by Kamal Bhupendra Lalloo

Sparse user–item interaction data remains one of the major challenges in recommender systems, often reducing recommendation quality for users with limited interaction histories. Although many recommendation approaches have been proposed, there is still limited systematic understanding of how different recommendation paradigms perform as interaction data becomes increasingly sparse.

This project evaluates multiple recommender system models, including classical collaborative filtering, neural recommendation, sequential recommendation, and graph-based approaches, under controlled sparsity conditions. Benchmark datasets are preprocessed into implicit-feedback interaction data, while sparsity is simulated through interaction reduction strategies such as global interaction removal, recent-history truncation, and cold-start-inspired profile construction.

Models are implemented and evaluated using the RecBole framework under standardized experimental settings to ensure fair comparison and reproducibility. Performance is assessed using ranking-based metrics including Recall@K, HitRate@K, NDCG@K, and MRR@K, together with robustness analysis based on performance degradation under progressively sparse conditions.

The goal is to provide a comparative analysis of recommender system robustness and improve understanding of how different recommendation approaches behave in realistic sparse-data environments.

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Research Questions	2
1.3	Research Aims and Objectives	2
1.4	Outline of the Research Methodology	3
1.5	Expected Contributions	3
2	Background and Related Work	4
2.1	Background	4
2.2	Related Work	5
2.3	Summary	9
3	Research Methodology	10
3.1	Dataset(s)	10
3.2	Preprocessing	11
3.2.1	Implicit Feedback Conversion	11
3.2.2	k-Core Filtering	11
3.2.3	Temporal Ordering	12
3.2.4	Data Splitting	12
3.2.5	Sparsity Simulation	12
3.3	Models and Algorithms Used	14
3.3.1	Baseline Model	14
3.3.2	Classical Collaborative Filtering Models	15
3.3.3	Linear Recommendation Model	15
3.3.4	Neural Recommendation Models	15
3.3.5	Sequential Recommendation Models	16
3.3.6	Graph-Based Recommendation Model	16
3.4	Evaluation Method	17
3.5	Summary	19
4	Work Schedule	20
4.1	Overview	20
4.2	Planned Activities	20
4.3	Project Timeline	21
4.4	Risk Management	21
4.5	Summary	21
5	Conclusion	22

Chapter 1

Introduction

Recommender systems are widely used on online platforms to provide personalized suggestions such as movies, products, music, and social media content. By learning patterns from historical user–item interactions, these systems help users navigate large volumes of information and improve user engagement.

Collaborative filtering is one of the most widely used recommendation approaches, with early methods such as item-based collaborative filtering and matrix factorization achieving strong performance through similarity modeling and latent representation learning [Sarwar et al., 2001; Koren et al., 2009]. In many practical applications, recommender systems rely on implicit feedback such as clicks, purchases, and viewing history rather than explicit ratings. Ranking-based approaches such as Bayesian Personalized Ranking (BPR) were introduced to optimize recommendation ranking under implicit-feedback settings [Hu et al., 2008; Rendle et al., 2009].

More recent approaches use deep learning, sequential modeling, and graph-based techniques to capture complex user behavior and higher-order interaction patterns. Models such as Neural Collaborative Filtering, SASRec, BERT4Rec, and LightGCN have demonstrated strong recommendation performance across various tasks [He et al., 2017; Kang and McAuley, 2018; Sun et al., 2019; He et al., 2020].

Despite these advances, data sparsity remains a major challenge in recommender systems. Users typically interact with only a small fraction of available items, limiting the information available for learning reliable recommendation patterns. This often leads to degraded recommendation quality, particularly for users with limited interaction histories and cold-start scenarios. In this study, robustness refers to the ability of a recommender system to maintain recommendation performance as available interaction data becomes increasingly sparse.

Although many approaches have been proposed to address sparse-data challenges, there remains limited systematic understanding of how different recommendation paradigms behave under varying levels of sparsity. Existing work often focuses on improving individual models rather than comparing their robustness under controlled sparse-data conditions. Furthermore, recent studies suggest that more complex deep learning approaches do not always consistently outperform simpler classical methods when evaluated fairly [Ferrari Dacrema et al., 2019].

This proposal investigates the robustness of recommender systems under sparse user-feedback conditions through controlled sparsity simulation on benchmark datasets. Multiple recommendation paradigms are evaluated to examine how recommendation performance changes as interaction data becomes increasingly sparse and which approaches remain most robust under realistic sparse-data environments.

1.1 Problem Statement

Recommender systems commonly operate under sparse interaction conditions where users engage with only a small fraction of available items. Limited interaction data reduces the information available for learning reliable user preferences and often leads to degraded recommendation quality, particularly for new users, infrequent users, and newly introduced items.

Although many recommender system models have been proposed to address sparse-data challenges, there remains limited systematic understanding of how different recommendation paradigms behave under varying levels of sparsity. Existing work often focuses on improving individual models rather than evaluating their robustness under controlled sparse-data conditions. Furthermore, recent studies suggest that more complex recommendation models do not always consistently outperform simpler approaches when evaluated fairly [Ferrari Dacrema et al., 2019].

These limitations motivate the need for a systematic evaluation of recommender systems under progressively sparse interaction conditions to better understand their robustness and practical performance.

1.2 Research Questions

This study is guided by the following research questions:

1. How does recommender system performance change as user-item interaction data becomes progressively sparse?
2. How do different recommendation model families perform when users have limited interaction histories?
3. Do modern neural, sequential, and graph-based recommendation models consistently outperform simpler classical recommendation approaches under sparse interaction conditions?

1.3 Research Aims and Objectives

Research Aim

The main aim of this study is to systematically evaluate and compare the robustness of recommender system models under varying levels of sparse user feedback.

Research Objectives

The objectives of this study are:

1. To preprocess benchmark recommendation datasets into implicit user–item interaction data suitable for recommendation tasks.
2. To simulate different sparse-data scenarios, including global sparsity, limited recent histories, and cold-start-like user profiles.
3. To implement and evaluate multiple recommender system models from different recommendation paradigms using a standardized evaluation framework.
4. To compare the performance of classical, neural, sequential, and graph-based recommendation models under varying levels of sparsity.
5. To evaluate recommendation performance using standard ranking-based metrics such as Recall@K, HitRate@K, NDCG@K, and MRR@K.
6. To analyse and compare the robustness of recommendation models as interaction data becomes increasingly sparse.

1.4 Outline of the Research Methodology

This study follows a controlled experimental methodology using benchmark recommendation datasets. The datasets are preprocessed into implicit-feedback interaction data and split chronologically into training, validation, and testing sets. Different sparsity scenarios are simulated by progressively reducing the amount of training interaction data available. Multiple recommender system models are then trained and evaluated under standardized experimental settings using the RecBole framework to ensure fair comparison and reproducibility. Performance is assessed using standard top-K ranking metrics, while robustness is measured through performance degradation under increasing sparsity conditions.

1.5 Expected Contributions

This study is expected to contribute:

- a systematic evaluation of recommender systems under controlled sparse-data conditions,
- a comparative analysis of multiple recommendation paradigms,
- and insights into how recommendation performance changes under varying levels of sparsity.

The remainder of this proposal is structured as follows. Chapter 2 reviews related work on recommender systems and sparse-data learning. Chapter 3 presents the research methodology, including datasets, preprocessing, sparsity simulation, model implementation, and evaluation methods. Chapter 4 outlines the project timeline and planned work schedule. Finally, Chapter 5 concludes the proposal and summarizes the expected contributions of the study.

Chapter 2

Background and Related Work

2.1 Background

Recommender systems aim to infer user preferences from historical user–item interactions. These interactions are commonly represented as a user–item interaction matrix, which is typically sparse because users interact with only a small subset of available items [Koren et al., 2009].

One of the foundational recommendation approaches is collaborative filtering, which assumes that users with similar past behavior are likely to share similar preferences in the future. Early methods such as item-based collaborative filtering estimate similarities between users or items using observed interaction patterns [Sarwar et al., 2001].

To model user preferences more effectively, matrix factorization techniques were introduced, which learn low-dimensional latent representations of users and items within a shared latent space [Koren et al., 2009]. Interactions are commonly modeled using the inner product:

$$\hat{r}_{ui} = p_u^T q_i$$

where p_u and q_i represent latent user and item embeddings respectively.

In many real-world applications, recommender systems rely on implicit feedback such as clicks, purchases, and viewing history rather than explicit ratings. Unlike explicit feedback, implicit interactions do not directly indicate preference strength and often lack clearly defined negative examples [Hu et al., 2008]. To address this, Bayesian Personalized Ranking (BPR) was introduced as a ranking-based optimization approach that encourages observed interactions to rank above unobserved ones [Rendle et al., 2009]:

$$\max_{\Theta} \sum_{(u,i,j) \in D_S} \ln \sigma(\hat{x}_{uij}) - \lambda_{\Theta} \|\Theta\|^2$$

These foundational approaches form the basis of many modern recommender systems. However, their effectiveness remains strongly influenced by the quality and sparsity of the available interaction data, motivating the development of more advanced recommendation models.

2.2 Related Work

Building on the foundational methods discussed in the previous section, a wide range of recommender system models have been proposed to address the limitations of traditional approaches. These developments include deep learning models that aim to capture more complex user–item relationships, sequential methods that incorporate temporal dynamics, and graph-based approaches that model higher-order interactions. In parallel, other lines of work have focused on improving learning under limited data conditions through techniques such as self-supervised learning and data augmentation. Despite these advances, challenges related to data sparsity and robustness remain, motivating continued research into the behavior of these models under realistic conditions.

Deep Learning for Recommendation

To address the limitations of linear models such as matrix factorization, recent work has explored deep learning approaches for modeling complex, non-linear relationships between users and items. One of the early examples is Neural Collaborative Filtering, which replaces the standard dot product interaction with a neural network to learn more flexible interaction patterns [He et al., 2017].

Another important direction is the use of probabilistic deep models such as variational autoencoders (VAEs), which learn latent representations of user preferences through non-linear generative modeling [Liang et al., 2018]. More broadly, deep learning techniques including multilayer perceptrons, recurrent networks, and autoencoders have been applied in recommender systems to learn richer representations and capture complex interaction patterns [Zhang et al., 2019].

Although deep learning models provide greater modeling flexibility than classical approaches, they remain sensitive to sparse interaction data and can be prone to overfitting due to their large number of parameters. As a result, they do not fully resolve the challenges associated with data sparsity.

Sequential Recommendation Models

Traditional recommender systems often model user preferences without explicitly considering the order of interactions. To address this limitation, sequential recommendation models were introduced, which aim to predict future user behavior based on interaction sequences. These models represent user activity as ordered sequences and capture dependencies between successive interactions, making them suitable for applications where preferences evolve over time.

One of the earlier approaches in this area uses recurrent neural networks (RNNs) to model user sessions. Session-based RNN models process sequences of interactions within a session and use this information to generate recommendations, particularly in cases where long-term user profiles are

unavailable [Hidasi et al., 2016]. Building on this idea, more recent methods adopt self-attention mechanisms. For example, SASRec uses attention to identify and weigh relevant past interactions when predicting the next item [Kang and McAuley, 2018]. Similarly, BERT4Rec extends this approach by using bidirectional self-attention, enabling the model to learn representations from both past and future context within a sequence [Sun et al., 2019].

Although sequential models capture temporal user behavior, their performance can still be affected by sparse or incomplete interaction histories, as shorter or less informative sequences may provide limited context for learning dependencies. As a result, handling sparsity remains an important consideration for sequential recommendation models.

Graph-Based Recommendation

Unlike sequential recommendation approaches, which focus on temporal dependencies, graph-based recommender systems model structural relationships between users and items. These systems represent user-item interactions as a graph, where users and items are treated as nodes while interactions form edges in a bipartite graph. This representation allows models to capture higher-order interaction patterns through shared neighbors [Wang et al., 2019].

One of the key models in this area is Neural Graph Collaborative Filtering (NGCF), which applies graph neural networks to propagate embeddings across the user-item interaction graph. Through iterative message passing, NGCF aggregates information from multi-hop neighbors, allowing embeddings to encode higher-order relationships between items and users [Wang et al., 2019]. Building on this approach, LightGCN simplifies the graph convolution framework by focusing on neighborhood aggregation and removing feature transformations and nonlinear activations. This design reduces model complexity while maintaining competitive recommendation performance [He et al., 2020].

Further developments focus on improving representation learning through additional training objectives. For example, Neural Contrastive Learning (NCL) introduces self-supervised learning techniques that leverage structural and semantic relationships within the interaction graph to improve learned representations under limited data conditions [Lin et al., 2022].

While graph-based approaches can partially compensate for limited direct interactions through neighborhood propagation, they remain dependent on the structure and quality of the interaction graph. Sparse or incomplete interaction data can limit graph connectivity and reduce the information available for propagation. As a result, graph-based models do not fully eliminate the challenges associated with sparse and noisy interaction data.

Handling Sparse Data and Cold Start

Data sparsity and cold-start scenarios remain key challenges in recommender systems, particularly when users or items have limited interaction history. To address these issues, a range of methods has been proposed to support learning under limited data conditions. Broadly, these approaches focus on incorporating additional information or generating additional training signals from existing data.

One common strategy is to combine collaborative filtering with auxiliary information. For example, DropoutNet simulates missing interactions during training using dropout, with the aim of improving performance in cold-start scenarios by incorporating both interaction data and side information [Volkovs et al., 2017]. Other approaches focus on data augmentation. Generative feedback models, for instance, simulate user interactions by creating synthetic feedback, providing additional training signals for learning from limited data [Wang et al., 2021].

In addition to model-specific approaches, survey work has examined broader categories of cold-start solutions, including content-based, collaborative, and hybrid methods [Glisovic et al., 2025]. However, no single method has been shown to fully address the challenges associated with sparse data and cold-start scenarios. The effectiveness of different methods depends on factors such as the availability of additional information, the structure of the data, and the application context.

Self-Supervised and Contrastive Learning

Recent work has explored self-supervised and contrastive learning as an approach to learning from limited interaction data. Rather than relying solely on observed user feedback, these methods generate additional training signals directly from the data, allowing models to learn from augmented or transformed views of the original interactions.

In sequential recommendation, S³-Rec introduces self-supervised pretraining tasks based on mutual information maximization to capture relationships between items, attributes, and sequences before optimizing the main recommendation objective [Zhou et al., 2020]. Similarly, CL4SRec applies contrastive learning by constructing augmented versions of user interaction sequences and training the model to distinguish between similar and dissimilar sequence views [Xie et al., 2021].

This idea has also been applied to graph-based recommendation models. For example, Self-supervised Graph Learning (SGL) creates multiple variations of the interaction graph and learns from these variations using contrastive learning [Wu et al., 2021]. Building on this, Neighborhood-enriched Contrastive Learning (NCL) incorporates both structural relationships from the graph and similarity between learned representations into the training objective [Lin et al., 2022].

Although these methods introduce additional training signals from the data itself, their effectiveness still depends on factors such as data characteristics and augmentation strategy, and they do not fully resolve sparse interaction challenges.

Robustness and Real-World Challenges

While many recommender models perform well under controlled conditions, their effectiveness can be affected in real-world settings due to missing data, noise, and bias. In practice, user interactions are often incomplete, inconsistent, and sometimes unreliable, which can impact model behavior. This highlights the importance of designing systems that are robust to imperfect data.

One key issue is the sensitivity of models to missing interactions, particularly in sequential recommendation. These models rely on patterns in user interaction sequences, and missing elements in a sequence can affect model predictions. Recent work shows that even small perturbations, such as removing individual interactions, can influence model outputs [Siciliano et al., 2024]. Robust training approaches attempt to improve prediction stability by accounting for missing inputs during learning. Similarly, models such as RETR focus on identifying more informative user behaviors through mechanisms such as pathway attention [Yao et al., 2022].

Noise in interaction data is another challenge. User actions such as accidental clicks or inconsistent behavior can introduce misleading signals into the learning process. Approaches such as graph-based filtering or pruning techniques attempt to reduce the influence of less informative interactions and improve model stability.

In addition to missing data and noise, recommender systems are also affected by popularity bias, where frequently interacted items are recommended more often than less popular ones. This can limit diversity and reinforce imbalances in the data, particularly in sparse settings where popular items dominate the interaction space. Prior work shows that recommender systems tend to over-represent popular items while under-representing long-tail items, which can reduce item discovery [Abdollahpouri et al., 2019].

Overall, these challenges indicate that real-world recommendation is influenced not only by model design but also by the quality of the underlying data. Handling missing data, reducing noise, and mitigating bias remain important considerations in developing reliable recommender systems.

Critical Perspectives and Model Comparison

Despite the rapid development of increasingly complex recommender system models, recent work has questioned whether these advances consistently lead to improved performance. In particular, concerns have been raised regarding evaluation practices in the literature and the extent to which deep learning approaches provide benefits over simpler methods.

A notable study by Dacrema et al. examines the performance of deep learning-based recommender models under consistent evaluation settings and reports that many do not consistently outperform well-tuned classical baselines [Ferrari Dacrema et al., 2019]. Their analysis shows that several deep learning approaches perform comparably to simpler methods such as nearest-neighbor and graph-based models, and in some cases even non-personalized methods remain competitive. The authors attribute these findings to factors such as weak baselines, inconsistent experimental setups, and limited reproducibility.

Similarly, the EASE-R model provides an example of a simple linear approach that achieves competitive performance relative to more complex methods [Steck, 2019]. Despite its simplicity, EASE-R has been reported to perform comparably to, and in some cases outperform, more complex collaborative filtering models on standard benchmarks. These findings indicate that increased model complexity does not necessarily lead to better results.

Overall, this line of work highlights the importance of careful evaluation, strong baselines, and consideration of data characteristics when comparing recommender system models, particularly under sparse-data conditions.

2.3 Summary

This chapter reviewed the development of recommender systems, from classical collaborative filtering methods to modern deep learning, sequential, and graph-based approaches. While classical approaches such as item-based filtering and matrix factorization remain strong baselines, their performance is limited in sparse data settings. More advanced models improve expressiveness by capturing non-linear relationships and temporal dynamics, but they remain dependent on sufficient interaction data.

Recent work has focused on mitigating sparsity through graph-based learning, self-supervised techniques, and cold-start strategies, aiming to extract more meaningful representations from limited data. However, challenges such as missing interactions, noise, and popularity bias continue to impact performance and generalization in real-world systems.

Importantly, prior studies show that increased model complexity does not necessarily lead to better performance, with simpler models often remaining competitive when properly evaluated [Ferrari Dacrema et al., 2019; Steck, 2019].

Overall, despite significant progress, the behavior of recommender systems under varying levels of data sparsity is not fully understood. This highlights the need for systematic evaluation to better assess model robustness and guide practical system design.

Chapter 3

Research Methodology

This chapter describes the methodology used to evaluate recommender system models under sparse user-feedback conditions. The study follows a controlled experimental approach in which multiple recommender models are trained and evaluated under progressively sparse interaction settings. The methodology includes dataset selection, preprocessing, sparsity simulation, model implementation, and evaluation using standard recommendation metrics.

The main objective of this study is to systematically compare how different recommendation approaches behave when interaction data becomes limited. In particular, the study focuses on measuring model robustness under sparse and cold-start-like scenarios.

3.1 Dataset(s)

This study uses two publicly available benchmark datasets commonly used in recommender system research: MovieLens-1M and Amazon Video Games 2023. The datasets were selected because they represent different recommendation environments and sparsity characteristics, allowing evaluation under both controlled and realistic sparse-data conditions. Both datasets contain timestamped user-item interactions, making them suitable for sequential recommendation tasks.

The datasets are preprocessed using the RecBole framework to ensure consistent formatting and evaluation across experiments.

TABLE 3.1: Dataset Statistics (Before Preprocessing)

Dataset	Users	Items	Interactions	Characteristics
MovieLens-1M	6,040	3,952	1,000,209	Relatively dense benchmark dataset
Amazon Video Games 2023	2.8M	137.2K	4.6M	Naturally sparse real-world dataset

Final dataset statistics after preprocessing and k-core filtering will be reported during experimentation.

MovieLens-1M

MovieLens-1M is a benchmark recommendation dataset developed by the GroupLens Research Group. The dataset contains explicit movie ratings together with timestamps indicating when interactions occurred. Compared to many real-world recommendation datasets, MovieLens-1M is relatively dense, making it suitable for controlled sparsity simulation experiments.

In this study, MovieLens-1M is primarily used as a controlled benchmark dataset for evaluating how recommendation performance changes as interactions are progressively removed.

Amazon Video Games 2023

The Amazon Video Games 2023 dataset contains user–item interactions collected from a real-world e-commerce platform. Unlike MovieLens-1M, the dataset is naturally sparse and contains many users and items with limited interaction histories.

The Video Games category was selected because it exhibits sparse user interaction behavior while containing timestamped implicit-feedback interactions suitable for sequential recommendation experiments. The dataset reflects more realistic recommendation environments where interaction data is noisy and incomplete.

Using both MovieLens-1M and Amazon Video Games 2023 allows evaluation across both controlled benchmark settings and realistic sparse recommendation environments.

3.2 Preprocessing

Before training and evaluation, the datasets undergo several preprocessing steps to ensure consistency and fair comparison across all experiments.

3.2.1 Implicit Feedback Conversion

Since the study focuses on implicit recommendation tasks, explicit ratings are converted into implicit feedback interactions. For the MovieLens-1M dataset, ratings greater than or equal to 4 are treated as positive interactions, while lower ratings are removed.

This converts the recommendation problem from explicit rating prediction into implicit-feedback ranking recommendation, where observed interactions are treated as positive feedback and unobserved interactions are treated as implicit negatives.

3.2.2 k-Core Filtering

A 5-core filtering strategy is applied to both datasets, where users and items with fewer than five interactions are removed. This filtering step is applied before sparsity simulation to establish a stable baseline dataset for controlled experimentation.

Although this study focuses on sparse user-feedback conditions, extremely sparse users and items are removed at this stage to reduce instability caused by insufficient interaction histories. Sparsity is then introduced in a controlled manner through the sparsity simulation procedures described later. This ensures that the observed performance changes are caused by the experimental sparsity conditions rather than by uncontrolled noise in the original datasets.

3.2.3 Temporal Ordering

Interactions are sorted chronologically using timestamps to create ordered user interaction sequences. Temporal ordering is important because sequential recommendation models depend on interaction order to learn user behavior patterns.

All preprocessing procedures are applied consistently across datasets using fixed random seeds to support reproducibility.

3.2.4 Data Splitting

A leave-one-out sequential splitting strategy is used for training, validation, and testing. For each user, the final interaction is used for testing, the second-last interaction is used for validation, and all earlier interactions are used for training.

Set	Interactions
Train	A, B, C
Validation	D
Test	E

This approach ensures that models are evaluated on future interactions, which better reflects real-world recommendation settings.

Full-sort evaluation is used during testing, where each model ranks all candidate items for a user rather than using sampled negative items. This provides a more realistic and consistent evaluation protocol across recommendation models.

3.2.5 Sparsity Simulation

To evaluate recommender system robustness under sparse interaction conditions, different sparsity scenarios are simulated by modifying only the training data while keeping validation and test sets unchanged. This ensures that all models are evaluated on the same validation and testing interactions across experiments.

Sparsity simulation is applied after preprocessing and chronological train-validation-test splitting. All sparsity experiments use fixed random seeds to support reproducibility. Users are required to retain sufficient interaction history for training, validation, and testing after sparsity simulation is applied.

Three sparsity scenarios are considered:

Global Sparsity

Global sparsity is simulated by uniformly removing interactions from the training set at random. Interaction removal is applied only to training interactions, while validation and test interactions remain unchanged throughout all experiments.

Different interaction retention levels are evaluated to simulate progressively sparse recommendation environments:

100%, 50%, 25%, 10%

For example, a 50% retention level retains only half of the original training interactions, while a 10% retention level retains only a small fraction of the available interaction data.

This sparsity scenario evaluates how recommendation performance changes as the overall interaction density decreases.

Recent-History Sparsity

Recent-history sparsity is simulated by truncating each user's interaction sequence to retain only the most recent interactions. Sequence truncation is applied independently per user while preserving chronological ordering.

Different retention levels are evaluated using the same interaction retention percentages:

100%, 50%, 25%, 10%

For example:

$$[A, B, C, D, E, F] \rightarrow [D, E, F]$$

This scenario simulates users with short recent interaction histories and evaluates how recommendation models behave when only limited recent behavioral information is available. The scenario is particularly relevant for sequential recommendation models that depend on recent interaction patterns.

Early-Profile Sparsity

Early-profile sparsity is simulated by retaining only the earliest interactions within each user's interaction sequence while removing more recent interactions. Truncation is applied independently per user while preserving chronological order.

Different interaction retention levels are evaluated using:

100%, 50%, 25%, 10%

For example:

$$[A, B, C, D, E, F] \rightarrow [A, B, C]$$

This scenario simulates cold-start-like recommendation environments where only limited early user behavior is available for learning user preferences. The experiment evaluates how recommendation models perform when long-term interaction histories are unavailable.

3.3 Models and Algorithms Used

This study evaluates recommender systems from multiple recommendation paradigms, including classical collaborative filtering, linear recommendation, neural recommendation, sequential recommendation, and graph-based recommendation models. The selected models represent different levels of architectural complexity and recommendation strategies commonly used in recommender system research.

Including both simple and complex approaches allows evaluation of whether increased model complexity consistently improves robustness under sparse interaction conditions. The selected models cover non-personalized, neighborhood-based, latent factor, neural, sequential, and graph-based recommendation approaches.

All models are implemented using the RecBole framework [Zhao et al., 2021], which provides standardized preprocessing, training, and evaluation procedures across multiple recommendation models. Using RecBole helps ensure reproducibility, consistent evaluation settings, and fair comparison across recommendation paradigms.

All experiments use identical preprocessing procedures, chronological train-validation-test splits, fixed random seeds, and standardized evaluation settings. Hyperparameter tuning is limited to constrained search spaces to maintain feasible experimental runtime while ensuring fair comparison across models.

3.3.1 Baseline Model

Popularity-Based Recommendation (Pop)

The popularity-based recommendation model recommends globally popular items based on overall interaction frequency. Although simple, the model provides an important non-personalized baseline because popular items often dominate sparse recommendation environments. The model is included

to evaluate the extent to which personalized recommendation approaches outperform popularity-driven recommendation under sparse conditions.

3.3.2 Classical Collaborative Filtering Models

ItemKNN

ItemKNN is an item-based collaborative filtering model that recommends items using similarity between historical user–item interaction patterns [Sarwar et al., 2001]. The model is included as a widely used and interpretable classical recommendation baseline that relies directly on observed interaction similarity.

Bayesian Personalized Ranking (BPR)

BPR is a matrix factorization model designed for implicit-feedback recommendation [Rendle et al., 2009]. Instead of predicting explicit preference scores, BPR optimizes pairwise ranking relationships between observed and unobserved items. The model is included as a standard latent factor baseline commonly used in implicit-feedback recommendation research.

3.3.3 Linear Recommendation Model

EASE

EASE (Embarrassingly Shallow Autoencoder) is a linear recommendation model designed for sparse interaction data [Steck, 2019]. The model reconstructs user–item interactions using a shallow linear formulation without deep neural network layers. EASE is included because prior work shows that simple linear recommendation models can achieve competitive performance relative to more complex neural architectures under standard benchmark evaluation settings.

3.3.4 Neural Recommendation Models

NeuMF

NeuMF is a neural collaborative filtering model that combines generalized matrix factorization with multilayer perceptrons to learn non-linear user–item interaction patterns [He et al., 2017]. The model is included as a representative deep learning-based collaborative filtering approach.

MultiVAE

MultiVAE is a variational autoencoder-based recommendation model that learns probabilistic latent representations of user preferences [Liang et al., 2018]. Variational recommendation approaches have been explored as a method for improving representation learning under sparse interaction conditions.

3.3.5 Sequential Recommendation Models

GRU4Rec

GRU4Rec is a recurrent neural network-based sequential recommender system that models temporal dependencies within user interaction sequences using gated recurrent units (GRUs) [Hidasi et al., 2016]. The model is included as a representative recurrent sequential recommendation approach.

SASRec

SASRec is a transformer-based sequential recommender system that uses self-attention mechanisms to identify relevant past interactions within user sequences [Kang and McAuley, 2018]. Compared to recurrent sequential models, transformer-based approaches improve the ability to model long-range interaction dependencies.

BERT4Rec

BERT4Rec extends transformer-based recommendation using bidirectional self-attention and masked-item prediction [Sun et al., 2019]. Unlike unidirectional sequential recommendation models, BERT4Rec learns contextual sequence representations using both past and future interactions within a sequence.

3.3.6 Graph-Based Recommendation Model

LightGCN

LightGCN is a graph collaborative filtering model that represents users and items as nodes within a bipartite interaction graph [He et al., 2020]. The model captures higher-order user-item relationships through graph propagation while simplifying traditional graph neural network operations by removing feature transformations and nonlinear activation functions. LightGCN is included as a representative graph-based recommendation approach commonly used in modern recommendation research.

TABLE 3.2: Selected Recommendation Models

Category	Model	Experimental Role
Baseline	Pop	Non-personalized popularity baseline used to measure the effectiveness of personalized recommendation approaches under sparse interaction conditions.
Classical Collaborative Filtering	ItemKNN	Neighborhood-based collaborative filtering model included as a traditional similarity-based recommendation baseline.
Classical Collaborative Filtering	BPR	Matrix factorization model optimized for implicit-feedback ranking recommendation tasks. Included as a standard latent factor baseline for sparse recommendation evaluation.
Linear Recommendation	EASE	Linear recommendation model designed for sparse interaction data. Included to evaluate whether simple linear approaches remain competitive relative to more complex architectures.
Neural Recommendation	NeuMF	Neural collaborative filtering model that learns non-linear user-item interaction patterns using multilayer perceptrons.
Neural Recommendation	MultiVAE	Variational autoencoder-based recommendation model that learns probabilistic latent user representations for implicit-feedback recommendation.
Sequential Recommendation	GRU4Rec	Recurrent neural network-based sequential recommendation model included as a representative recurrent sequence modeling approach.
Sequential Recommendation	SASRec	Transformer-based sequential recommendation model that uses self-attention to capture long-range interaction dependencies within user sequences.
Sequential Recommendation	BERT4Rec	Bidirectional transformer-based sequential recommendation model that learns contextual sequence representations using masked-item prediction.
Graph-Based Recommendation	LightGCN	Graph collaborative filtering model that captures higher-order user-item relationships through graph propagation within a bipartite interaction graph.

3.4 Evaluation Method

The recommendation models are evaluated using standard top-K ranking metrics commonly used in recommender system research. These metrics are selected because the study focuses on implicit-feedback ranking recommendation, where the relative ranking position of recommended items is more important than predicting explicit rating values.

Evaluation is performed using full-sort ranking, where each model ranks all candidate items for a user during testing rather than using sampled negative items. Full-sort evaluation is used to provide

fair and consistent comparison across recommendation models and to avoid inconsistencies introduced by negative sampling strategies.

The study evaluates recommendation performance using Recall@K, HitRate@K, NDCG@K, and MRR@K, where:

$$K \in \{5, 10, 20\}$$

Using multiple K values allows evaluation of recommendation quality across different recommendation list lengths.

- **Recall@K** measures the proportion of relevant items that appear within the top-K recommended items. Higher Recall@K values indicate better retrieval performance and improved ability to recover relevant items from the candidate item space.
- **HitRate@K** measures whether at least one relevant item appears within the top-K recommendation list. This metric evaluates recommendation success at the user level and reflects whether the recommender system is capable of producing at least one useful recommendation for a user.
- **NDCG@K** (Normalized Discounted Cumulative Gain) evaluates ranking quality by assigning greater importance to correctly ranked items appearing near the top of the recommendation list. NDCG@K is used as the primary evaluation metric because recommendation ranking position is important in practical recommendation systems where users are more likely to interact with highly ranked recommendations.
- **MRR@K** (Mean Reciprocal Rank) evaluates the ranking position of the first relevant recommended item. Higher MRR@K values indicate that relevant items appear earlier in the recommendation ranking, reflecting improved recommendation prioritization.

Robustness Evaluation

In addition to recommendation accuracy, the study evaluates model robustness under sparse interaction conditions. Robustness is measured using relative performance degradation between the full interaction dataset and sparse-data conditions:

$$\text{Performance Drop} = \frac{\text{Score}_{full} - \text{Score}_{sparse}}{\text{Score}_{full}}$$

where Score_{full} represents model performance using the full interaction dataset and Score_{sparse} represents model performance under simulated sparse conditions.

A smaller performance drop indicates a more robust recommender model. This robustness measure allows comparison not only of absolute recommendation accuracy, but also of how gracefully recommendation models degrade as interaction data becomes increasingly sparse.

To improve reproducibility and reduce sensitivity to random initialization effects, experiments are conducted using fixed random seeds and standardized evaluation settings across all recommendation models. Where computationally feasible, experiments are repeated across multiple random seeds and average performance values are reported.

3.5 Summary

This chapter presented the methodology used to evaluate recommender systems under sparse user-feedback conditions. The study uses benchmark datasets, controlled sparsity simulation, and multiple recommendation paradigms to systematically evaluate model robustness.

Different recommendation models, including classical collaborative filtering, neural recommendation, sequential recommendation, and graph-based recommendation approaches, are evaluated under varying sparsity scenarios using standard ranking metrics.

The methodology is designed to provide a fair and reproducible comparison of recommender system performance under realistic sparse-data environments.

All experiments are conducted using fixed random seeds and standardized evaluation settings to support reproducibility and fair comparison across recommendation models.

Chapter 4

Work Schedule

4.1 Overview

This chapter presents the planned work schedule for the proposed study. The project follows a structured experimental workflow consisting of dataset preparation, sparsity simulation, recommender system implementation, evaluation, robustness analysis, and dissertation writing.

The project timeline spans approximately four months, from late June 2026 to mid October 2026. Earlier phases focus on dataset preprocessing, implementation validation, and baseline recommendation experiments, while later phases focus on advanced recommendation experiments, robustness analysis, dissertation writing, and final revisions.

4.2 Planned Activities

The project is divided into four main phases corresponding to the experimental workflow of the study.

- **Environment Setup and Dataset Preparation**

This phase includes RecBole configuration, dataset acquisition, preprocessing, implicit-feedback conversion, k-core filtering, temporal ordering, train-validation-test splitting, and implementation of sparsity simulation procedures.

- **Recommendation Experiments**

Baseline recommendation models are first evaluated to validate the experimental pipeline. More advanced neural, sequential, and graph-based recommendation models are then evaluated under different sparsity scenarios using standardized experimental settings.

- **Evaluation and Robustness Analysis**

Experimental results are analysed using Recall@K, HitRate@K, NDCG@K, and MRR@K. Additional analysis focuses on measuring performance degradation under sparse interaction conditions and comparing robustness across recommendation paradigms.

- **Dissertation Writing and Final Preparation**

The final phase focuses on dissertation writing, result interpretation, revision, proofreading,

formatting, and final corrections prior to submission. Additional time is reserved for debugging, rerunning experiments if necessary, and incorporating supervisor feedback.

4.3 Project Timeline

Table 4.1 presents the planned project timeline and major activities for the study.

TABLE 4.1: Project Timeline

Time Period	Planned Activities
Late June 2026	Environment setup, RecBole configuration, dataset acquisition, and preprocessing implementation
Early to Mid July 2026	Temporal splitting, sparsity simulation implementation, and evaluation pipeline validation
Mid to Late July 2026	Baseline recommendation experiments using Pop, ItemKNN, BPR, and EASE
August 2026	Neural, sequential, and graph-based recommendation experiments under multiple sparsity conditions
September 2026	Evaluation, robustness analysis, result visualization, and dissertation writing
Early to Mid October 2026	Dissertation revision, proofreading, final corrections, and submission preparation

4.4 Risk Management

One potential challenge is the computational cost associated with sequential and graph-based recommendation models such as BERT4Rec and LightGCN under multiple sparsity scenarios. To manage this, experiments will use constrained hyperparameter tuning, standardized evaluation settings, and limited training epochs to maintain feasible computational requirements. GPU acceleration will also be used where available.

Another potential challenge involves preprocessing, implementation, or debugging issues. To reduce these risks, experiments are conducted incrementally, beginning with simpler baseline models before progressing to more computationally expensive architectures. Additional time is reserved near the end of the project schedule to accommodate unexpected delays, rerunning experiments if necessary, and incorporating supervisor feedback.

4.5 Summary

This chapter presented the planned work schedule for the proposed study. The schedule is designed to support systematic experimentation, controlled evaluation, and sufficient time for dissertation writing, revision, and final preparation.

Chapter 5

Conclusion

This proposal presented a study investigating the robustness of recommender systems under sparse user-feedback conditions. Recommender systems commonly operate on highly sparse user-item interaction data, which can negatively affect recommendation quality, particularly for users with limited interaction histories and cold-start-like scenarios.

The proposal reviewed existing recommender system approaches, including classical collaborative filtering, neural recommendation, sequential recommendation, and graph-based recommendation models. Prior work related to sparse-data learning, robustness, and recommendation evaluation was also discussed to provide background for the proposed study.

To address the research problem, the study proposes a controlled experimental methodology using benchmark recommendation datasets and multiple sparsity simulation strategies. The proposed experiments aim to systematically evaluate how different recommendation paradigms behave as interaction data becomes increasingly sparse. Model performance will be evaluated using standard ranking-based metrics including Recall@K, HitRate@K, NDCG@K, and MRR@K, together with robustness analysis based on performance degradation under sparse conditions.

The proposed research is expected to provide a comparative analysis of recommender system robustness under varying sparse-data environments and contribute insights into the behavior of different recommendation approaches when interaction information is limited. Overall, the study aims to improve understanding of how recommender systems perform under realistic sparse-feedback conditions and support the development of more reliable recommendation strategies.

Bibliography

- [Abdollahpouri et al., 2019] Abdollahpouri, H., Burke, R., and Mobasher, B. (2019). Managing popularity bias in recommender systems with personalized re-ranking. In *FLAIRS Conference*.
- [Ferrari Dacrema et al., 2019] Ferrari Dacrema, M., Cremonesi, P., and Jannach, D. (2019). Are we really making much progress? a worrying analysis of recent neural recommendation approaches. In *ACM Conference on Recommender Systems (RecSys)*, pages 101–109.
- [Glisovic et al., 2025] Glisovic, N., Kragic, D., and Tegner, M. (2025). Item cold start in e-commerce recommender systems: A survey. *IEEE Access*, 13.
- [He et al., 2020] He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., and Wang, M. (2020). Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference*, pages 639–648.
- [He et al., 2017] He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. (2017). Neural collaborative filtering. In *Proceedings of the 26th International World Wide Web Conference (WWW)*, pages 173–182.
- [Hidasi et al., 2016] Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. (2016). Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*.
- [Hu et al., 2008] Hu, Y., Koren, Y., and Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. In *IEEE International Conference on Data Mining (ICDM)*, pages 263–272.
- [Kang and McAuley, 2018] Kang, W.-C. and McAuley, J. (2018). Self-attentive sequential recommendation. In *IEEE International Conference on Data Mining (ICDM)*, pages 197–206.
- [Koren et al., 2009] Koren, Y., Bell, R., and Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8):30–37.
- [Liang et al., 2018] Liang, D., Krishnan, R. G., Hoffman, M. D., and Jebara, T. (2018). Variational autoencoders for collaborative filtering. In *World Wide Web Conference (WWW)*, pages 689–698.
- [Lin et al., 2022] Lin, Z., Tian, C., Hou, Y., and Zhao, W. X. (2022). Improving graph collaborative filtering with neighborhood-enriched contrastive learning. In *ACM Web Conference (WWW)*.

- [Rendle et al., 2009] Rendle, S., Freudenthaler, C., Gantner, Z., and Schmidt-Thieme, L. (2009). Bpr: Bayesian personalized ranking from implicit feedback. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 452–461.
- [Sarwar et al., 2001] Sarwar, B., Karypis, G., Konstan, J., and Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In *World Wide Web Conference (WWW)*, pages 285–295.
- [Siciliano et al., 2024] Siciliano, F., Lagziel, S., Gamzu, I., and Tolomei, G. (2024). Robust training of sequential recommender systems with missing input data. In *RobustRecSys Workshop at RecSys*.
- [Steck, 2019] Steck, H. (2019). Embarrassingly shallow autoencoders for sparse data. In *World Wide Web Conference (WWW)*.
- [Sun et al., 2019] Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., and Jiang, P. (2019). Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1441–1450.
- [Volkovs et al., 2017] Volkovs, M., Yu, G., and Poutanen, T. (2017). Dropoutnet: Addressing cold start in recommender systems. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [Wang et al., 2021] Wang, W., Xu, H., Zhang, R., Wang, W., Rai, P., and Carin, L. (2021). Learning to recommend from sparse data via generative user feedback. In *AAAI Conference on Artificial Intelligence*.
- [Wang et al., 2019] Wang, X., He, X., Wang, M., Feng, F., and Chua, T.-S. (2019). Neural graph collaborative filtering. In *ACM SIGIR Conference*, pages 165–174.
- [Wu et al., 2021] Wu, J., Wang, X., Feng, F., He, X., Chen, L., Lian, J., and Xie, X. (2021). Self-supervised graph learning for recommendation. In *ACM SIGIR Conference*, pages 726–735.
- [Xie et al., 2021] Xie, X., Sun, F., Liu, Z., Wu, S., Gao, J., Ding, B., and Cui, B. (2021). Contrastive learning for sequential recommendation. In *ACM SIGIR Conference*.
- [Yao et al., 2022] Yao, Z., Chen, X., Wang, S., Dai, Q., Li, Y., Zhu, T., and Long, M. (2022). Recommender transformers with behavior pathways. *arXiv preprint arXiv:2206.06804*.
- [Zhang et al., 2019] Zhang, S., Yao, L., Sun, A., and Tay, Y. (2019). Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1).
- [Zhao et al., 2021] Zhao, W. X., Mu, S., Hou, Y., Lin, Z., Chen, Y., Pan, X., Li, K., Lu, Y., Wang, H., Tian, C., Min, Y., Feng, Z., Fan, X., Chen, X., Wang, P., Ji, W., Li, Y., Wang, X., and Wen, J.-R. (2021). Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 4653–4664.
- [Zhou et al., 2020] Zhou, K., Wang, H., Zhao, W. X., Zhu, Y., Wang, S., Zhang, F., Wang, Z., and Wen, J.-R. (2020). S³-rec: Self-supervised learning for sequential recommendation. In *ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1893–1902.